

Damerau–Levenshtein distance

From Wikipedia, the free encyclopedia

In information theory and computer science, the **Damerau–Levenshtein distance** (named after Frederick J. Damerau and Vladimir I. Levenshtein^{[1][2][3]}) is a string metric for measuring the edit distance between two sequences. Informally, the Damerau–Levenshtein distance between two words is the minimum number of operations (consisting of insertions, deletions or substitutions of a single character, or transposition of two adjacent characters) required to change one word into the other.

The Damerau–Levenshtein distance differs from the classical Levenshtein distance by including transpositions among its allowable operations in addition to the three classical single-character edit operations (insertions, deletions and substitutions).^{[4][2]}

In his seminal paper,^[5] Damerau stated that these four operations correspond to more than 80% of all human misspellings. Damerau's paper considered only misspellings that could be corrected with at most one edit operation. While the original motivation was to measure distance between human misspellings to improve applications such as spell checkers, Damerau–Levenshtein distance has also seen uses in biology to measure the variation between protein sequences.^[6]

Contents

- 1 Definition
- 2 Algorithm
 - 2.1 Optimal string alignment distance
 - 2.2 Distance with adjacent transpositions
- 3 Applications
 - 3.1 DNA
 - 3.2 Fraud detection
 - 3.3 Export control
- 4 See also
- 5 References
- 6 Further reading

Definition

To express the Damerau–Levenshtein distance between two strings ***a*** and ***b*** a function $d_{a,b}(i, j)$ is defined, whose value is a distance between an *i*–symbol prefix (initial substring) of string ***a*** and a *j*–symbol prefix of ***b***.

The function is defined recursively as:

$$d_{a,b}(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} d_{a,b}(i-1, j) + 1 \\ d_{a,b}(i, j-1) + 1 \\ d_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{if } i, j > 1 \text{ and } a_i = b_{j-1} \text{ and } a_{i-1} = b_j \\ \min \begin{cases} d_{a,b}(i-1, j) + 1 \\ d_{a,b}(i, j-1) + 1 \\ d_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

where $1_{(a_i \neq b_j)}$ is the indicator function equal to 0 when $a_i = b_j$ and equal to 1 otherwise.

Each recursive call matches one of the cases covered by the Damerau–Levenshtein distance:

- $d_{a,b}(i-1, j) + 1$ corresponds to a deletion (from a to b).
- $d_{a,b}(i, j-1) + 1$ corresponds to an insertion (from a to b).
- $d_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)}$ corresponds to a match or mismatch, depending on whether the respective symbols are the same.
- $d_{a,b}(i-2, j-2) + 1$ corresponds to a transposition between two successive symbols.

The Damerau–Levenshtein distance between a and b is then given by the function value for full strings: $d_{a,b}(|a|, |b|)$ where $i = |a|$ denotes the length of string a and $j = |b|$ is the length of b .

Algorithm

Presented here are two algorithms: the first,^[7] simpler one, computes what is known as the *optimal string alignment distance* or *restricted edit distance*,^[8] while the second one^[9] computes the Damerau–Levenshtein distance with adjacent transpositions. Adding transpositions adds significant complexity. The difference between the two algorithms consists in that the *optimal string alignment algorithm* computes the number of edit operations needed to make the strings equal under the condition that **no substring is edited more than once**, whereas the second one presents no such restriction.

Take for example the edit distance between **CA** and **ABC**. The Damerau–Levenshtein distance $LD(\mathbf{CA}, \mathbf{ABC}) = 2$ because $\mathbf{CA} \rightarrow \mathbf{AC} \rightarrow \mathbf{ABC}$, but the optimal string alignment distance $OSA(\mathbf{CA}, \mathbf{ABC}) = 3$ because if the operation $\mathbf{CA} \rightarrow \mathbf{AC}$ is used, it is not possible to use $\mathbf{AC} \rightarrow \mathbf{ABC}$ because that would require the substring to be edited more than once, which is not allowed in OSA, and therefore the shortest sequence of operations is $\mathbf{CA} \rightarrow \mathbf{A} \rightarrow \mathbf{AB} \rightarrow \mathbf{ABC}$. Note that for the optimal string alignment distance, the triangle inequality does not hold: $OSA(\mathbf{CA}, \mathbf{AC}) + OSA(\mathbf{AC}, \mathbf{ABC}) < OSA(\mathbf{CA}, \mathbf{ABC})$, and so it is not a true metric.

Optimal string alignment distance

Optimal string alignment distance can be computed using a straightforward extension of the Wagner–Fischer dynamic programming algorithm that computes Levenshtein distance. In pseudocode:

```

algorithm OSA-distance is
  input: strings a[1..length(a)], b[1..length(b)]
  output: distance, integer

  let d[0..length(a), 0..length(b)] be a 2-d array of integers, dimensions length(a)+1, length(b)+1
  // note that d is zero-indexed, while a and b are one-indexed.

  for i := 0 to length(a) inclusive do
    d[i, 0] := i
  for j := 0 to length(b) inclusive do
    d[0, j] := j

  for i := 1 to length(a) inclusive do
    for j := 1 to length(b) inclusive do
      if a[i] = b[j] then
        cost := 0
      else
        cost := 1
      d[i, j] := minimum(d[i-1, j] + 1,      // deletion
                        d[i, j-1] + 1,      // insertion
                        d[i-1, j-1] + cost) // substitution
      if i > 1 and j > 1 and a[i] = b[j-1] and a[i-1] = b[j] then
        d[i, j] := minimum(d[i, j],
                           d[i-2, j-2] + cost) // transposition
  return d[length(a), length(b)]

```

The difference from the algorithm for Levenshtein distance is the addition of one recurrence:

```

if i > 1 and j > 1 and a[i] = b[j-1] and a[i-1] = b[j] then
    d[i, j] := minimum(d[i, j],
                      d[i-2, j-2] + cost) // transposition

```

Distance with adjacent transpositions

The following algorithm computes the true Damerau–Levenshtein distance with adjacent transpositions; this algorithm requires as an additional parameter the size of the alphabet Σ , so that all entries of the arrays are in $[0, |\Sigma|)$:

```

algorithm DL-distance is
  input: strings a[1..length(a)], b[1..length(b)]
  output: distance, integer

  da := new array of |Σ| integers
  for i := 1 to |Σ| inclusive do
    da[i] := 0

  let d[-1..length(a), -1..length(b)] be a 2-d array of integers, dimensions length(a)+2, length(b)+2
  // note that d has indices starting at -1, while a, b and da are one-indexed.

  maxdist := length(a) + length(b)
  d[-1, -1] := maxdist
  for i := 0 to length(a) inclusive do
    d[i, -1] := maxdist
    d[i, 0] := i
  for j := 0 to length(b) inclusive do
    d[-1, j] := maxdist
    d[0, j] := j

  for i := 1 to length(a) inclusive do
    db := 0
    for j := 1 to length(b) inclusive do
      k := da[b[j]]
      ℓ := db
      if a[i - 1] = b[j - 1] then
        cost := 0
        db := j
      else
        cost := 1
        d[i, j] := minimum(d[i-1, j-1] + cost, //substitution
                          d[i, j-1] + 1,      //insertion
                          d[i-1, j] + 1,      //deletion
                          d[k-1, ℓ-1] + (i-k-1) + 1 + (j-ℓ-1)) //transposition

      da[a[i]] := i
    return d[length(a), length(b)]

```

To devise a proper algorithm to calculate unrestricted Damerau–Levenshtein distance note that there always exists an optimal sequence of edit operations, where once-transposed letters are never modified afterwards. (This holds as long as the cost of a transposition, W_T , is at least the average of the cost of an insertion and deletion, i.e., $2W_T \geq W_I + W_D$.^[9]) Thus, we need to consider only two symmetric ways of modifying a substring more than once: (1) transpose letters and insert an arbitrary number of characters between them, or (2) delete a sequence of characters and transpose letters that become adjacent after deletion. The straightforward implementation of this idea gives an algorithm of cubic complexity: $O(M \cdot N \cdot \max(M, N))$, where M and N are string lengths. Using the ideas of Lowrance and Wagner,^[9] this naive algorithm can be improved to be $O(M \cdot N)$ in the worst case.

It is interesting that the bitap algorithm can be modified to process transposition. See the information retrieval section of^[1] for an example of such an adaptation.

Applications

Damerau–Levenshtein distance plays an important role in natural language processing. In natural languages, strings are short and the number of errors (misspellings) rarely exceeds 2. In such circumstances, restricted and real edit distance differ very rarely. Oommen and Loke^[7] even mitigated the limitation of the restricted edit distance by introducing *generalized transpositions*. Nevertheless, one must remember that the restricted edit distance usually does not satisfy the triangle inequality and, thus, cannot be used with metric trees.

DNA

Since DNA frequently undergoes insertions, deletions, substitutions, and transpositions, and each of these operations occurs on approximately the same timescale, the Damerau–Levenshtein distance is an appropriate metric of the variation between two strands of DNA. More common in DNA, protein, and other bioinformatics related alignment tasks is the use of closely related algorithms such as Needleman–Wunsch algorithm or Smith–Waterman algorithm.

Fraud detection

The algorithm can be used with any set of words, like vendor names. Since entry is manual by nature there is a risk of entering a false vendor. A fraudster employee may enter one real vendor such as "Rich Heir Estate Services" versus a false vendor "Rich Hier State Services". The fraudster would then create a false bank account and have the company route checks to the real vendor and false vendor. The Damerau–Levenshtein algorithm will detect the transposed and dropped letter and bring attention of the items to a fraud examiner.

Export control

The U.S. Government uses the Damerau–Levenshtein distance with its Consolidated Screening List API.^[10]

See also

- Ispell suggests corrections that are based on a Damerau–Levenshtein distance of 1
- Typosquatting

References

1. Brill, Eric; Moore, Robert C. (2000). *An Improved Error Model for Noisy Channel Spelling Correction* (<https://web.archive.org/web/20121221172057/http://acl.ldc.upenn.edu/P/P00/P00-1037.pdf>) (PDF). Proceedings of the 38th Annual Meeting on Association for Computational Linguistics. pp. 286–293. doi:10.3115/1075218.1075255 (<https://doi.org/10.3115%2F1075218.1075255>). Archived from the original (<http://acl.ldc.upenn.edu/P/P00/P00-1037.pdf>) (PDF) on 2012-12-21.
2. Bard, Gregory V. (2007), "Spelling-error tolerant, order-independent pass-phrases via the Damerau–Levenshtein string-edit distance metric", *Proceedings of the Fifth Australasian Symposium on ACSW Frontiers : 2007, Ballarat, Australia, January 30 - February 2, 2007* (<http://dl.acm.org/citation.cfm?id=1274531.1274545>), Conferences in Research and Practice in Information Technology, **68**, Darlinghurst, Australia: Australian Computer Society, Inc., pp. 117–124, ISBN 1-920-68285-6. The isbn produces two hits: a 2007 work and a 2010 work at World Cat.
3. Li; et al. (2006). *Exploring distributional similarity based models for query spelling correction* (<https://web.archive.org/web/20100401081500/http://acl.ldc.upenn.edu:80/P/P06/P06-1129.pdf>) (PDF). Proceedings of the 21st International Conference on Computational Linguistics and the 44th annual meeting of the Association for Computational Linguistics. pp. 1025–1032. doi:10.3115/1220175.1220304 (<https://doi.org/10.3115%2F1220175.1220304>). Archived from the original (<http://acl.ldc.upenn.edu/P/P06/P06-1129.pdf>) (PDF) on 2010-04-01.
4. Levenshtein, Vladimir I. (February 1966), "Binary codes capable of correcting deletions, insertions, and reversals", *Soviet Physics Doklady*, **10** (8): 707–710

5. Damerau, Fred J. (March 1964), "A technique for computer detection and correction of spelling errors", *Communications of the ACM*, ACM, **7** (3): 171–176, doi:10.1145/363958.363994 (<https://doi.org/10.1145/363958.363994>)
6. The method used in: Majorek, Karolina A.; Dunin-Horkawicz, Stanisław; et al. (2013), "The RNase H-like superfamily: new members, comparative structural analysis and evolutionary classification" (<http://nar.oxfordjournals.org/content/42/7/4160.full>), *Nucleic Acids Research*, **42** (7): 4160–4179, PMC 3985635 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3985635>) , PMID 24464998 (<https://www.ncbi.nlm.nih.gov/pubmed/24464998>), doi:10.1093/nar/gkt1414 (<https://doi.org/10.1093/nar/gkt1414>)
7. Oommen, B. J.; Loke, R. K. S. "Pattern recognition of strings with substitutions, insertions, deletions and generalized transpositions". *Pattern Recognition*. **30**: 789–800. CiteSeerX 10.1.1.50.1459 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.50.1459>) , doi:10.1016/S0031-3203(96)00101-X ([https://doi.org/10.1016/S0031-3203\(96\)00101-X](https://doi.org/10.1016/S0031-3203(96)00101-X)).
8. Boytsov, Leonid (May 2011). "Indexing methods for approximate dictionary searching". *Journal of Experimental Algorithmics*. Association for Computing Machinery (ACM). **16**: 1. doi:10.1145/1963190.1963191 (<https://doi.org/10.1145/1963190.1963191>).
9. Lowrance, Roy; Wagner, Robert A. (April 1975), "An Extension of the String-to-String Correction Problem", *JACM*, **22** (2): 177–183, doi:10.1145/321879.321880 (<https://doi.org/10.1145/321879.321880>)
10. <http://developer.trade.gov/consolidated-screening-list.html>

Further reading

- Navarro, Gonzalo (March 2001), "A guided tour to approximate string matching", *ACM Computing Surveys*, **33** (1): 31–88, doi:10.1145/375360.375365 (<https://doi.org/10.1145/375360.375365>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Damerau–Levenshtein_distance&oldid=793134337"

-
- This page was last edited on 30 July 2017, at 21:25.
 - Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.